

JAVA PROGRAMMING

ASSIGNMENT 12

1. Bank Account Management

- **Problem Description:**
 - Create a `BankAccount` class with attributes such as `accountNumber`, `accountHolderName`, and `balance`. Implement methods to deposit money, withdraw money, and check the balance. Create multiple `BankAccount` objects and perform various transactions (like deposit and withdrawal). Display the final balance for each account.
- **Solution Approach:**
 - Start by defining the `BankAccount` class with the required attributes and methods. Use a loop or individual calls to simulate transactions (deposit/withdrawal) for each account. Finally, print out the account details and balance.
- **Test Cases:**
 1. **Test Case 1:**
 - **Input:** Deposit 5000, Withdraw 2000.
 - **Expected Outcome:** Final balance = 3000.
 2. **Test Case 2:**
 - **Input:** Deposit 10000, Withdraw 15000.
 - **Expected Outcome:** Error message (Insufficient balance).
 3. **Test Case 3:**
 - **Edge Case:** Initial balance = 0, Withdraw 100.
 - **Expected Outcome:** Error message (Insufficient balance).
 4. **Test Case 4:**
 - **Edge Case:** Deposit 0.
 - **Expected Outcome:** Balance remains unchanged.
 5. **Test Case 5:**
 - **Input:** Multiple deposits and withdrawals.
 - **Expected Outcome:** Final balance reflects all transactions accurately.

2. Library System

- **Problem Description:**
 - Design a `Book` class with attributes `title`, `author`, and `isbn`. Create a `Library` class that manages a collection of `Book` objects. Implement methods to add new books, remove books by ISBN, and list all books.
- **Solution Approach:**
 - Start by creating the `Book` class, then build the `Library` class with an internal list to store `Book` objects. Use methods to manage the collection (add, remove, list). Test the system by adding/removing multiple books.
- **Test Cases:**
 1. **Test Case 1:**
 - **Input:** Add three books with unique ISBNs.
 - **Expected Outcome:** Library lists all three books.
 2. **Test Case 2:**
 - **Input:** Remove a book by its ISBN.
 - **Expected Outcome:** Book is removed from the list.
 3. **Test Case 3:**
 - **Edge Case:** Try removing a book not in the library.
 - **Expected Outcome:** Error message (Book not found).
 4. **Test Case 4:**
 - **Input:** Add a book with an already existing ISBN.
 - **Expected Outcome:** Error message (Duplicate ISBN).
 5. **Test Case 5:**
 - **Input:** List books when the library is empty.
 - **Expected Outcome:** Message indicating no books available.

3. Contact Management

- **Problem Description:**
 - Create a `Contact` class with attributes `name`, `phoneNumber`, and `email`. Implement methods to update contact details and display contact information. Create a `ContactManager` class that maintains a list of `Contact` objects. Implement methods to add contacts, search for a contact by name, and list all contacts.
- **Solution Approach:**
 - Define the `Contact` class first. Then, create the `ContactManager` class with methods for adding, updating, and searching contacts. Implement logic for listing all contacts in the manager.
- **Test Cases:**
 1. **Test Case 1:**
 - **Input:** Add three contacts.
 - **Expected Outcome:** All three contacts are listed.
 2. **Test Case 2:**
 - **Input:** Search for a contact by name.
 - **Expected Outcome:** Correct contact details are displayed.
 3. **Test Case 3:**
 - **Edge Case:** Search for a non-existent contact.
 - **Expected Outcome:** Message indicating contact not found.
 4. **Test Case 4:**
 - **Input:** Update an existing contact's phone number.
 - **Expected Outcome:** Updated details are displayed.
 5. **Test Case 5:**
 - **Input:** Add a contact with a duplicate name.
 - **Expected Outcome:** Allow adding but differentiate by unique email/phone.

4. Vehicle and Owner

- **Problem Description:**
 - Design a `Vehicle` class with attributes `make`, `model`, and `year`. Create an `Owner` class with attributes `name`, `address`, and a list of `Vehicle` objects owned. Implement methods to add vehicles to an owner's list and display owner details along with their vehicles.
- **Solution Approach:**
 - Define the `Vehicle` class and the `Owner` class. The `Owner` class should have methods to manage the list of vehicles. Simulate the process of adding vehicles and displaying owner information.
- **Test Cases:**
 1. **Test Case 1:**
 - **Input:** Add two vehicles to an owner.
 - **Expected Outcome:** Owner's information displays both vehicles.
 2. **Test Case 2:**
 - **Input:** Add a vehicle with incomplete details.
 - **Expected Outcome:** Error or prompt to complete information.
 3. **Test Case 3:**
 - **Edge Case:** Display an owner with no vehicles.
 - **Expected Outcome:** Owner details are displayed with no vehicles listed.
 4. **Test Case 4:**
 - **Input:** Remove a vehicle from the list.
 - **Expected Outcome:** Updated list of vehicles for the owner.
 5. **Test Case 5:**
 - **Input:** Add multiple vehicles with the same model and year.
 - **Expected Outcome:** All vehicles are added and displayed correctly.

5. Student Grades

- **Problem Description:**
 - Create a `Student` class with attributes `name` and a list of grades. Implement methods to add grades, calculate the average grade, and display student details. Create a `GradeBook` class that manages a list of `Student` objects. Implement methods to add students to the grade book, find the student with the highest average grade, and display all students' information.
- **Solution Approach:**
 - Start with the `Student` class, implementing methods to handle grades and calculations. Then, create the `GradeBook` class to manage multiple students and their grades. Use loops or sorting to determine the highest average grade.
- **Test Cases:**
 1. **Test Case 1:**
 - **Input:** Add three students with different grades.
 - **Expected Outcome:** All students' details are listed with correct averages.
 2. **Test Case 2:**
 - **Input:** Find the student with the highest average grade.
 - **Expected Outcome:** The correct student is identified.
 3. **Test Case 3:**
 - **Edge Case:** Add a student with no grades.
 - **Expected Outcome:** Average calculation should handle or return zero.
 4. **Test Case 4:**
 - **Input:** Add grades for an existing student.
 - **Expected Outcome:** Grades are added, and the average is recalculated.
 5. **Test Case 5:**
 - **Input:** List all students when the grade book is empty.
 - **Expected Outcome:** Message indicating no students available.